

Enrutamiento en Angular

Desarrollo Web en Entorno Cliente

Daniel Suárez Martín

Curso 25/26

Índice

Contenido

Índice	2
Introducción	3
1. Creación de la página principal	3
2. Componente 'Inicio'	5
3. Configuración y lógica en 'EvolucionesBulbasaur'	5
4. Componentes 'Ivysaur', 'Venusaur' y 'Error404'	8
5. Comprobación y capturas	8
CONCLUSIÓN	11

Introducción

Esta última práctica de la asignatura consistirá en poner en práctica los conocimientos adquiridos sobre el sistema de enrutamiento en aplicaciones desarrolladas con Angular. A través de esta actividad se trabajará con la configuración de rutas y la navegación entre diferentes componentes dentro de una misma aplicación, permitiendo comprender el funcionamiento del Angular *Router* y su integración en la estructura del proyecto.

El objetivo principal será desarrollar una pequeña aplicación web que permita visualizar la línea evolutiva del Pokémon Bulbasaur y sus evoluciones: Ivysaur y Venusaur. Para ello, se crearán distintos componentes que representen cada etapa evolutiva y se establecerán diferentes rutas que permitirán navegar entre ellas mediante enlaces, botones e interacciones con imágenes.

Además, se implementarán rutas dinámicas que interpretarán el nivel del Pokémon a partir de la URL introducida por el usuario, mostrando automáticamente la evolución correspondiente según el rango de niveles establecido. De esta forma, se pretende reforzar la comprensión del uso de parámetros en las rutas y la gestión de la navegación dentro de una aplicación Angular.

El proyecto, debido al tamaño que tiene incluso tras ser comprimido, ha sido publicado en Github para tener acceso al mismo. Se puede revisar todo el programa a través [del siguiente enlace al repositorio de la práctica](#).

1. Creación de la página principal

Como hemos visto en prácticas anteriores, al lanzar nuestra aplicación, el archivo inicial 'index.html' lanza por defecto en primer 'app.html' dentro de las etiquetas 'app-root'. Hasta ahora, en 'app.html' simplemente colocábamos las etiquetas necesarias para lanzar los distintos componentes de nuestra aplicación. Como en este caso se nos pide añadir un 'navbar' y un 'footer', los desarrollaremos en este apartado para que estén continuamente cargados en nuestra aplicación y no tener que configurarlo en ninguno de nuestros componentes. Además, dado que vamos a utilizar enrutamientos, la etiqueta para llamar a los componentes en este caso será 'router-outlet', y no la del nombre de los componentes.

```
</nav>
<router-outlet></router-outlet>
<footer class=" bg-success py-3 fixed-bottom">
  <div class="container">
    <div class="row">
      <h6 class="text-center">Página web realizada por Daniel Suárez Martín</h6>
    </div>
  </div>
</footer>
```

Como se puede apreciar en la imagen, usaremos el *framework* Bootstrap para dotar de estilo a nuestra aplicación. Para ello, modificaremos, igual que en la práctica anterior, el fichero 'angular.json'.

El siguiente paso será configurar el fichero 'app.ts' para que el enrutamiento funcione correctamente. Para ello, importaremos 'RouterLink' y 'RouterOutlet'.

```
import { Component, signal } from '@angular/core';
import { RouterLink, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
```

El último paso para la configuración del enrutamiento consistirá en modificar el archivo 'app.router.ts'. Este es el paso más importante, pues será donde definamos qué componente se cargará en cada URL de nuestra aplicación.

Para ello, importaremos los distintos componentes creados para nuestra aplicación, y a continuación indicaremos en el array de rutas qué URL le pertenece a cada componente:

```
import { Routes } from '@angular/router';
import { Inicio } from './components/inicio/inicio';
import { EvolucionesBulbasaur } from './components/evolucionesbulbasaur/evolucionesbulbasaur';
import { Ivysaur } from './components/ivysaur/ivysaur';
import { Venusaur } from './components/venusaur/venusaur';
import { Error404 } from './components/error404/error404';

export const routes: Routes = [
  {path: "", component: Inicio},
  {path: "bulbasaur", component: EvolucionesBulbasaur},
  {path: "Ivysaur", component: Ivysaur},
  {path: "Venusaur", component: Venusaur},
  {path: "bulbasaur/:id", component: EvolucionesBulbasaur},
  {path: "**", component: Error404}
];
```

Como se puede comprobar en la imagen, el string correspondiente a cada 'path' es dicha URL. Además de los indicados mediante cadenas literales, tenemos el primero ("") que es nuestro punto de inicio de la web, el penúltimo ("bulbasaur/:id") que utiliza los caracteres introducidos tras la

barra para cargar una parte del componente de 'EvolucionesBulbasaur' (lo veremos más adelante), y el último ("**"), que hace referencia a cualquier URL que no esté definida anteriormente, que en mi caso cargará un componente de Error 404 con un botón de vuelta a la URL inicial.

2. Componente 'Inicio'

Como hemos observado en la anterior captura, el componente que abre por defecto nuestra aplicación al lanzarse es 'Inicio'. En este simplemente hemos desarrollado un pequeño texto de bienvenida, y un botón que nos mandará a la URL `"/bulbasaur"`. En este caso, en lugar de utilizar un `'href'` dentro de la etiqueta `'a'` de nuestro botón, usaremos `'routerLink'`, pues lo que estamos haciendo es cargar componentes mediante enrutamiento de la URL. Para que este funcione, deberemos importar `'RouterLink'` en el archivo `'inicio.ts'`, de la misma forma que lo hicimos en `'app.ts'`.

```
<div class="container py-5">
  <div class="row text-center">
    <h1>¡Bienvenido a la ruta evolutiva de Bulbasaur!</h1>
    <div class="py-5">
      <a routerLink="/bulbasaur" class="btn btn-success btn-lg mt-4 px-4">¡Comenzar!</a>
    </div>
  </div>
</div>
```

3. Configuración y lógica en 'EvolucionesBulbasaur'

Al pulsar el botón de `¡Comenzar!`, se nos dirigirá a la URL `"/bulbasaur"`, la cual carga el componente `'EvolucionesBulbasaur'`. Este se trata del componente más importante de la práctica, pues dado que, además de sus respectivas URLs, según el ID que acompañe a `"/bulbasaur"` haremos que este evolucione o no, deberemos manejar esa lógica interna dentro de este componente. Para ello, nos valdremos de las directivas vistas en temas anteriores.

Para todo esto, en nuestro fichero `'ts'` de este componente, además de importar `'RouterLink'`, también lo haremos con `'CommonModule'`, y nuevamente implementaremos `'OnInit'` en nuestra clase (al igual que en la anterior práctica), solo que en este caso, dado que vamos a usar el ID de la URL para mostrar una información u otra, el método `'ngOnInit()'` no estará vacío, sino que será aquí donde indiquemos que se cargará una información u otra en función de los datos introducidos:

```
ngOnInit(): void {
  this.route.paramMap.subscribe(params => {
    const id = params.get("id");
    this.cargarDatos(id);
  })
}
```

Como observamos, esto hará que se cargue el método `'cargarDatos'` pasando dicha ID como parámetro. Esta función lo que hará será modificar ciertos datos en función del parámetro recogido.

Los datos a modificar, en este caso, serán las variables usadas para mostrar la descripción del Pokémon, la imagen cargada, y el valor de un objeto llamado `'pokemon'`, que

usaremos más adelante el en fichero 'html' de este componente para modificar la apariencia del mismo.

```
descripcion: string = "<b>Bulbasaur</b> <br><br> Tras nacer, crece alimentándose  
img: string = "assets/img/bulbasaur.png";  
pokemon: 'bulbasaur' | 'ivysaur' | 'venusaur' | null = 'bulbasaur';
```

```
cargarDatos(id: string | null){  
  if (!id){  
    this.pokemon = 'bulbasaur';  
    return;  
  }  
  let numeroId: number = parseInt(id, 10);  
  if(numeroId >= 1 && numeroId <=15){  
    this.pokemon = 'bulbasaur';  
    return;  
  }  
  if(numeroId >= 16 && numeroId <=31){  
    this.descripcion = "<b>Ivysaur</b> <br><br>";  
    this.img = "assets/img/ivysaur.png";  
    this.pokemon = 'ivysaur';  
    return;  
  }  
  if(numeroId >= 32 && numeroId <=100){  
    this.descripcion = "<b>Venusaur</b> <br><br>";  
    this.img = "assets/img/venusaur.png";  
    this.pokemon = 'venusaur';  
    return;  
  }  
  if(numeroId > 100 || numeroId < 1){  
    this.descripcion = "El nivel de un Poké";  
    this.pokemon = null;  
    return;  
  }  
}
```

Como se observa en la imagen de la izquierda, nuestro método, en primer lugar, comprueba la existencia o no de dicho ID. A continuación, parsea dicho ID a número para comprobar si se encuentra en los rangos válidos para cada fase evolutiva de Bulbasaur (1-15, Bulbasaur, 16-31, Ivysaur, 32-100, Venusaur).

En caso de salir de rango, se mostrará una información que indica que el rango introducido no es válido.

A continuación, en nuestro archivo 'html' de este componente, utilizaremos:

- **Directiva 'ngIf'** para cargar la imagen, descripción, y botones adecuados en función a la fase evolutiva correspondiente.

```
<!-- Imagen -->
<a *ngIf="pokemon === 'bulbasaur'" routerLink="/bulbasaur/16">
  <img [src]="img" alt="imagen dinámica" class="img-fluid w-100">
</a>
<a *ngIf="pokemon === 'ivysaur'" routerLink="/bulbasaur/32">
  <img [src]="img" alt="imagen dinámica" class="img-fluid w-100">
</a>
<img *ngIf="pokemon === 'venusaur'" [src]="img" alt="imagen dinámica" class="img-fluid w-100">
```

- **Enrutamiento** a través de la imagen de la página (que nos mandará a una URL que use la ID para evolucionar a la fase de Bulbasaur correspondiente) y en los botones bajo la imagen (que nos llevarán la URL que activa el componente de cada fase evolutiva).

```
<!-- Botones -->
<div class="d-grid gap-2 my-4">
  <!-- Evolución Bulbasaur -->
  <a *ngIf="pokemon === 'bulbasaur'"
    routerLink="/Ivysaur"
    class="btn btn-success">
    Evolucionar
  </a>
  <!-- Ivysaur -->
  <a *ngIf="pokemon === 'ivysaur'"
    routerLink="/bulbasaur"
    class="btn btn-success">
    Retroceder
  </a>
  <a *ngIf="pokemon === 'ivysaur'"
    routerLink="/Venusaur"
    class="btn btn-success">
    Evolucionar
  </a>
  <!-- Venusaur -->
  <a *ngIf="pokemon === 'venusaur'"
    routerLink="/Ivysaur"
    class="btn btn-success">
    Retroceder
  </a>
</div>
</div>
```

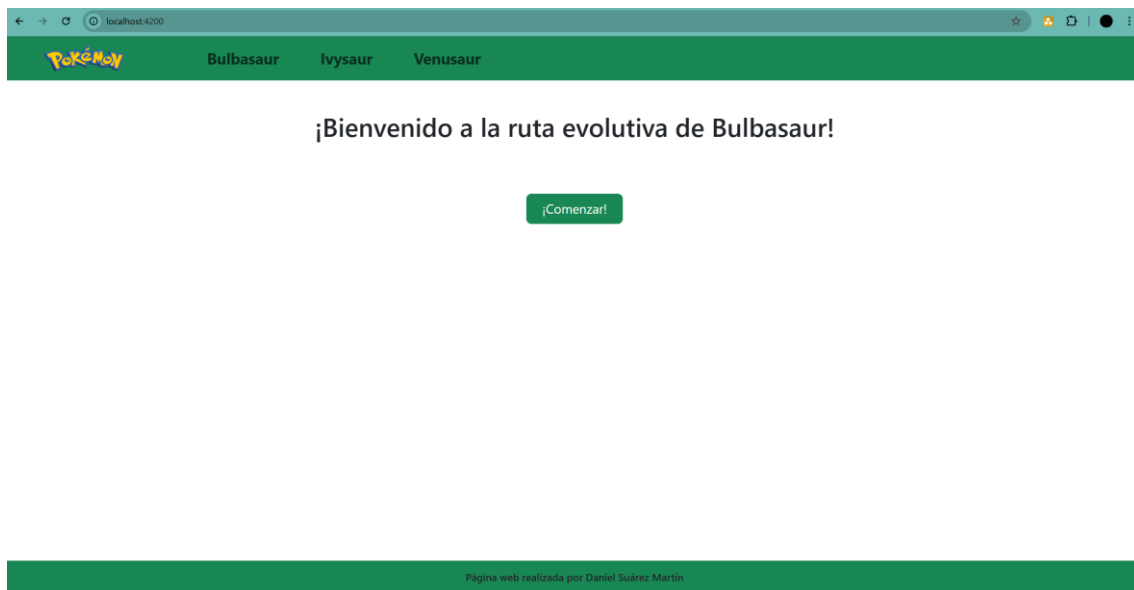
4. Componentes 'Ivysaur', 'Venusaur' y 'Error404'

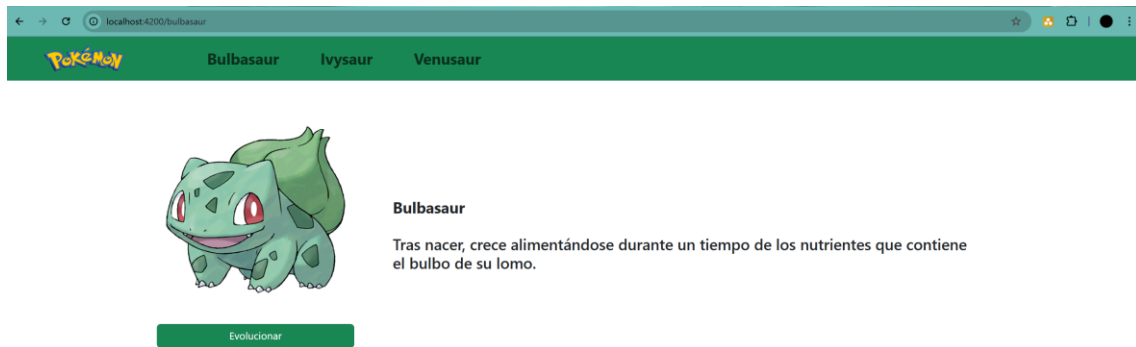
Dado que estos componentes, al cargarse, siempre mostrarán la misma información, podemos utilizar información en plano dentro de sus respectivos 'html' para mostrar su imagen, descripción y configurar el enrutamiento de los botones. Este enrutamiento sí que precisará, nuevamente, de importar la clase 'RouterLink' dentro de cada fichero 'ts'.

```
<div class="container py-5">
  <div class="row text-center">
    <h1>ERROR 404</h1>
    <p class="py-4"><b>La URL introducida no existe o está en construcción.</b></p>
    <div>
      <a routerLink="" class="btn btn-primary btn-lg mt-4 px-4">Volver a la página de Inicio</a>
    </div>
  </div>
</div>
```

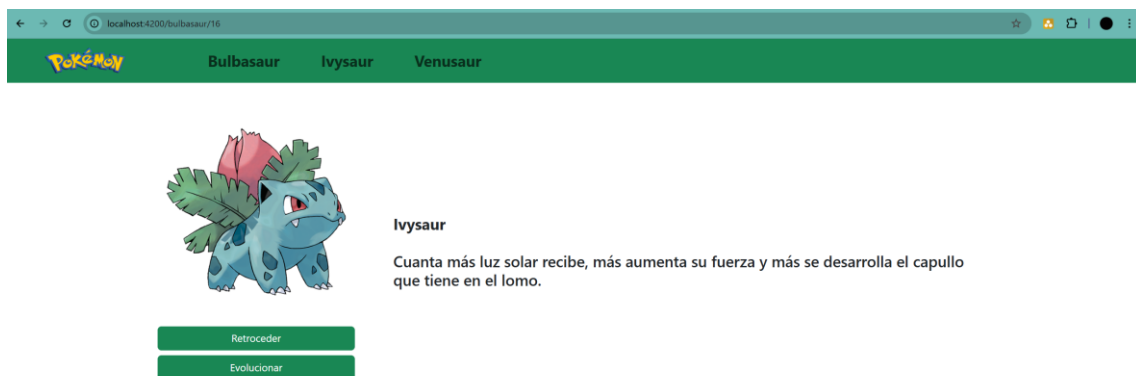
5. Comprobación y capturas

Una vez hemos terminado de desarrollar la lógica interna, lanzamos, mediante comandos en terminal, el comando 'ng serve -o' para comprobar el correcto funcionamiento de nuestra aplicación:

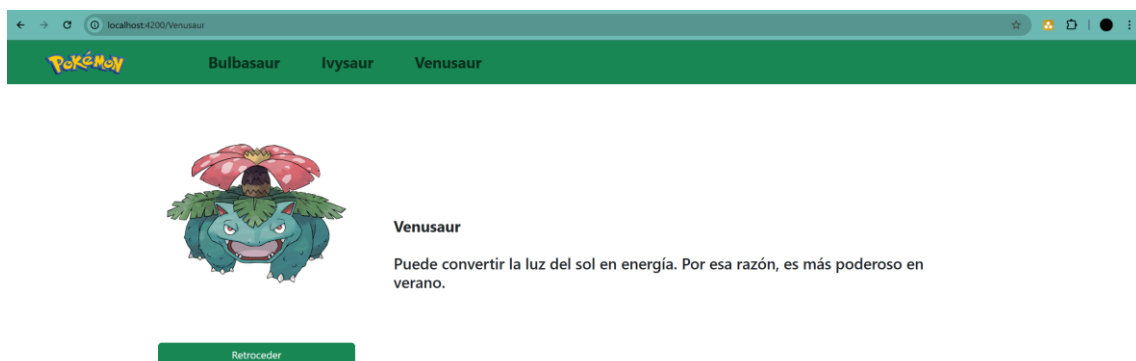




Página web realizada por Daniel Suárez Martín



Página web realizada por Daniel Suárez Martín



Página web realizada por Daniel Suárez Martín



El nivel de un Pokémon debe estar entre 1 y 100.



ERROR 404

La URL introducida no existe o está en construcción.

[Volver a la página de Inicio](#)



CONCLUSIÓN

En esta práctica se ha desarrollado una pequeña aplicación utilizando Angular con el objetivo de aplicar los conocimientos relacionados con el sistema de enrutamiento dentro de una aplicación web. A lo largo del desarrollo se han configurado distintas rutas que permiten navegar entre varios componentes, así como rutas dinámicas capaces de interpretar parámetros incluidos en la URL.

Para ello, se ha implementado una aplicación que muestra la línea evolutiva del Pokémon Bulbasaur y sus evoluciones Ivysaur y Venusaur, utilizando distintos componentes y controlando la información mostrada en función del nivel introducido en la dirección web. Esto ha permitido trabajar con directivas estructurales como `'ngIf'`, así como con el uso de `'routerLink'` y `'router-outlet'` para gestionar la navegación entre vistas.

Además, se ha reforzado la comprensión de la estructura de un proyecto Angular, la organización de los componentes y la lógica necesaria para mostrar contenido dinámico en función de los datos recibidos desde la URL. Gracias a ello, se ha podido comprobar cómo Angular permite desarrollar aplicaciones web estructuradas y fácilmente escalables mediante el uso de su sistema de componentes y enrutamiento.

Finalmente, tras realizar las pruebas correspondientes mediante la ejecución del proyecto en el servidor de desarrollo, se ha verificado que la aplicación funciona correctamente y que todas las rutas y funcionalidades implementadas responden de la forma esperada.